

10 HATEOAS

Príklad 8: Portál Svet kníh - varianta s použitím štýlu HATEOAS

Cvičenia

10 HATEOAS

V predchádzajúcej časti sme si ilustrovali viacero problémov, s ktorými sa môžeme stretnúť pri návrhu, implementácii a používaní REST služby poskytujúcej prístup komplexnému systému navzájom asociovaných zdrojov. Aby sme sa problémom vyhli, implementovali sme GET metódy tak, že reprezentácia zdroja s odkazmi na ďalšie, asociované zdroje neobsahuje celú reprezentáciu asociovaných zdrojov, ale len ich identifikátory. Na základe týchto identifikátorov si môže klient vytvoriť URI asociovaných zdrojov a pristupovať k nim priamo.

Elegantnejšie riešenie by bolo poslať v odpovedi namiesto identifikátora asociovaného zdroja priamo jeho URI. URI asociovaných objektov môžeme vložiť do hlavičky http správy ako link. Asociované zdroje, ani ich identifikátory by sa do obsahu správy (vo formáte XML resp. JSON) vôbec neserializovali.

V slučve port8lu SvetKnih by sme mohli GET metódu pre URI `SvetKnih/{obchod}` implementovať takto:

Príklad 8: Portál Svet kníh - varianta s použitím štýlu HATEOAS

```
@GET
@Path("/{obchod}")
@Produces(MediaType.APPLICATION_XML)
public Response getObchod(@Context UriInfo uriInfo,
    @PathParam("obchod") String obchod) {
    Knihkupectvo kk = em.find(Knihkupectvo.class, obchod);
    if (kk!=null && kk.getPonuka()!=null) {
        Response.ResponseBuilder rb = Response.ok(kk);
        for (Kniha k : kk.getPonuka()) {
            rb = rb.link(uriInfo.getBaseUriBuilder()
                .path("SvetKnih")
                .path("kniha")
                .path(k.getIsbn()).build()
                , "ponuka");
        }
        return rb.build();
    }
    return null;
}
```

Metóda vracia inštanciu `Response`, reprezentujúcu HTTP správu s odpoveďou. Dátový objekt kníhkupectva je serializovaný ako obsah správy.

Pre každú knihu v ponuke vloží do hlavičky správy link s URI na priamy prístup ku knihe. Link okrem URI obsahuje aj ďalší identifikátor, ktorý vyjadruje vzťah k referencovanému zdroju. Tu sme použili `"ponuka"`, keďže ide o knihu v ponuke kníhkupectva.

Poznámka: Link možno vložiť aj priamo do správy vo formáte XML.

Pozri napríklad <https://www.baeldung.com/spring-hateoas-tutorial>

Rovnakým spôsobom môžeme upraviť aj ďalšie GET metódy:

- GET metódu pre `SvetKnih/kniha/{isbn}`, ktorá vracia reprezentáciu knihy. Tu do hlavičky odpovede vložíme URI kníhkupectiev, v ktorých je kniha v ponuke.
- GET metóda pre `SvetKnih` vráti v hlavičke URI všetkých obchodov, namiesto zoznamu názvov obchodov v obsahu správy.

- GET metóda pre `SvetKnih/kniha` vráti v hlavičke URI hľadaných kníh, namiesto zoznamu ich ISBN v obsahu správy.

HATEOAS – Hypermedia as the Engine of Application State.

Štýl implementácie REST služby, ktorý sme tu použili, sa nazýva HATEOAS. Rieši problém s cyklickými referenciami. Klientovi umožňuje postupne pristupovať k asociovaným objektom ako samostatným zdrojom tak, že do obsahu správy vo formáte XML (resp. JSON) zapíše len dátové členy jednoduchých dátových typov. Asociované objekty predstavujú samostatné zdroje, ich URI sa posielajú ako link. Reprezentáciu asociovaného zdroja si musí klient potom vyžiadať ďalšou GET požiadavkou pre jeho URI.

Implementáciu služby portálu **SvetKníh** s využitím štýlu HATEOAS nájdete v projekte [SvetKnihV4](#).

Implementácia klienta

Projekt [SvetKnihV4klient](#) obsahuje jednoduchú klientskú aplikáciu, ktorá nájde všetky kníhkupectvá, pre každé vypíše jeho názov a všetky informácie o knihách v ponuke.

```
Client client = ClientBuilder.newClient();
Response resp = client
    .target("http://localhost:8080/SvetKnihV4/resources")
    .path("SvetKnih")
    .request(MediaType.TEXT_PLAIN)
    .get(Response.class);

for(Link l: resp.getLinks()) {
    Response resp2 = client
        .target(l)
        .request(MediaType.APPLICATION_XML)
        .get(Response.class);
    String obchod = resp2.readEntity(String.class);
    System.out.println(obchod);

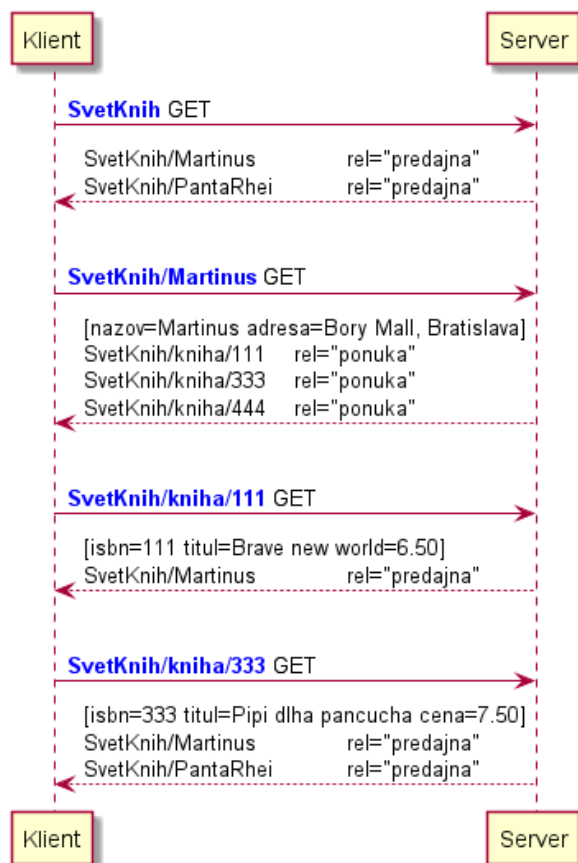
    for(Link l2: resp2.getLinks()) {
        String kniha = client
            .target(l2)
            .request(MediaType.APPLICATION_XML)
            .get(String.class);
        System.out.println(kniha);
    }
}
```

Klient pošle najprv GET požiadavku pre koreňový zdroj `SvetKnih`. Návratovú správu načíta do inštancie `Response`, aby mal prístup k hlavičke, ktorá obsahuje URI asociovaných zdrojov t. j. kníhkupectiev.

Potom pre každé kníhkupectvo vyšle GET požiadavku a odpoveď opäť načíta do inštancie `Response`. Odpoveď teraz obsahuje reprezentáciu kníhkupectva (názov, adresu...) a v hlavičke má URI asociovaných zdrojov, teda kníh v ponuke.

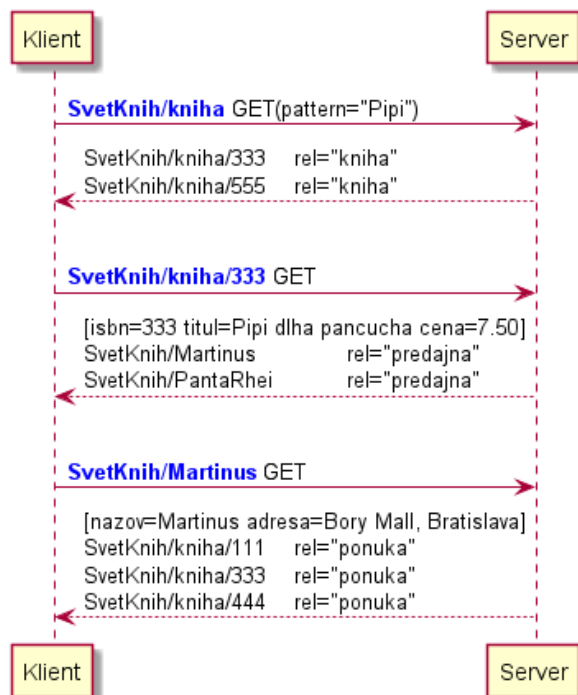
Ďalšou GET požiadavkou získa jej reprezentáciu knihy, ktorú načíta tentoraz ako už textový reťazec.

Celú komunikáciu medzi serverom a klientom, ktorý číta informácie o kníhkupectvách a ponúkaných knihách, ilustruje **sekvenčný diagram 1**:



Portál SvetKnih však umožňuje medzi zdrojmi navigovať aj inak.

Napríklad klient, ktorý si chce kúpiť konkrétnu knihu, si ju môže najprv vyhľadať podľa názvu, zistiť si jej cenu a potom aj adresu kníhkupectva, kde ju možno kúpiť. Celú komunikáciu klienta pri hľadaní konkrétnej knihy zobrazuje **sekvenčný diagram 2**:



Pripomeňme, že WADL súbor neobsahuje opis vnútornej štruktúry dát posielaných vo formáte XML alebo JSON. Ak táto štruktúra nie je známa pri tvorbe klientskej aplikácie, nemôžeme pre ne vytvoriť vhodnú dátovú triedu, umožňujúcu pohodlný prístup k jednotlivým dátovým položkám (cene, adrese...).

Štýl HATEOAS môžeme však aplikovať aj na jednoduché dátové členy. Štruktúrované dáta nie sú potom reprezentované vo formáte XML resp. JSON, ale každý dátový člen (nielen štruktúrovaný) je prístupný ako samostatný zdroj. Jeho URI sa posiela ako link, pričom názov dátovej položky je identifikátor vzťahu. Posielajú sa teda len textové správy. A tak klient už nemusí mať vopred žiadne vedomosti o štruktúre dát spravovaných serverom. Odhaľuje ju postupne na základe linkov v odpovediach od servera. Týmto spôsobom sa teda vyrieši aj problém tvorby klientskej aplikácie, keď štruktúra zdrojov nie je klientovi vopred známa.

HATEOAS je štýl implementácie REST služby, oddeľujúci klienta od serveru spôsobom, ktorý **umožňuje serveru nezávislé rozširovanie funkcionality**.

Reimplementáciu služby portálu SvetKnih s využitím štýlu HATEOAS tak, že sa posielajú len jednoduché textové správy, ako aj klientov prenechávame na **cvičenie 10.3**.

Komunikáciu medzi takto implementovanou službou a klientom, ktorý si chce kúpiť konkrétnu knihu, zobrazuje **sekvenčný diagram 3**:



Cvičenia

Cvičenie 10.1 Implementácia klientov portálu Svet kníh

Implementujte klientskú aplikáciu služby z príkladu 8 (projekt [SvetKnihV4](#)), ktorá vyhľadá kníhkupectvo, v ktorom majú knihu o Pipi dlhej pančuche. Komunikácia by mala prebiehať tak, ako zobrazuje **sekvenčný diagram 2**.

Cvičenie 10.2

Rovnako ako v príklade 7 (projekt [SvetKnihV3](#)), umožňuje aj služba z príkladu 8 (projekt [SvetKnihV4](#)) klientom pridať novú knihu do portálu. Nová kniha ale nebude v ponuke žiadneho kníhkupectva.

Navrhnite a implementujte aj pre túto variantu služby rozšírenie, ktoré by umožňovalo klientom pridať do ponuky kníhkupectva knihu, ktorá existuje už v portáli.

Navrhnite a implementujte pre túto variantu služby rozšírenie, ktoré by umožňovalo klientom aj odstrániť knihu z ponuky kníhkupectva.

Cvičenie 10.3

Implementujte službu portálu SvetKnih, tak že štýl HATEOAS aplikujete na všetky dátové členy tried Knihkupectvo a Kniha.

Implementujte tiež klientskú aplikáciu, ktorá vyhľadá informácie o knihe 'Pipi dlhá pančucha a obchode, kde ju možno kúpiť'. Komunikácia medzi klientom a službou by mala prebiehať tak, ako zobrazuje **sekvenčný diagram 3**.

Cvičenie 10.4 Implementácia služby jedálne

Reimplementujte službu jedálne z **cvičenia 8.10** s využitím štýlu HATEOAS.

Implementujte tiež klientskú aplikáciu služby jedálne, ktorá zistí a vypíše všetky informácie o jedlách v ponuke na pondelok.